

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering
Jackfruit Phase-1 | UE24CS341A – Software Engineering

SOFTWARE REQUIREMENTS SPECIFICATION

for SnakeArcade – Configurable Desktop Snake Game

Item	Details
Project Title	SnakeArcade
Project ID / Group	SG-2026-01 / Team 1 (G-01)
Version	1.0 (Requirements Baseline)
Prepared by	Rohil Thimmanahalli Girish, Sai Harshith R, Sarayu Mayakonda
Course	UE24CS341A – Software Engineering, V Semester
Date	September 2026

Revision History

Version	Date	Description
1.0	September 2026	Initial requirements baseline prepared from the approved Synopsis / Project Proposal, covering game session management, snake movement, food/growth/scoring, collision handling, pause/resume, difficulty selection and high-score persistence.

1. Introduction

1.1 Purpose

This document specifies the software requirements for SnakeArcade, a single-player desktop Snake game built in C++ using the SFML graphics library. It covers the complete scope of the game as planned for this course submission: game-session management, gameplay mechanics, difficulty selection, scoring and persistent high-score storage. This SRS is the requirements baseline against which the architecture, design, implementation and test cases for the project will be validated. Scope is intentionally kept lean and directly traceable to the approved Synopsis's functional feature list, so that requirement, design, code and test effort stay proportionate to a 3-person, one-semester team.

1.2 Intended Audience and Reading Suggestions

This document is intended for: (a) the three development-team members, who will use it to derive design, code and test cases; (b) the course instructor/evaluator, who will use it to assess requirement quality and traceability; and (c) any future maintainer of the codebase. Readers should start with Sections 1–2 for context, Section 5 for the detailed functional requirements, and Appendix C for how each requirement maps through to code and tests.

1.3 Product Scope

SnakeArcade is a self-contained, single-player arcade game. Its purpose is to provide an enjoyable, replayable game session while giving the development team a realistic, moderately complex codebase on which to practice a full software-engineering lifecycle: requirements, architecture/design, Agile implementation, automated testing, CI/CD and validation. It has no dependency on any external system; the only persistent data is the locally stored all-time high score.

1.4 References

- PES University Jackfruit Mini-Project — Software Engineering Project Guidelines, 2026 (UE24CS341A).
- PES University SE Level-3 Student Submission Guide & Coffee Kiosk Exemplar, 2026.
- Prescribed Software Requirements Specification template (Jackfruit Phase-1).
- SFML 2.6 Official Documentation — <https://www.sfm1-dev.org/documentation/>

- Team 1 Synopsis / Project Proposal, SnakeArcade, v1.0.

2. Overall Description

2.1 Product Perspective

SnakeArcade is a new, self-contained desktop application; it is not a follow-on to an existing system and has no external customer. It runs entirely on a single machine with no server or network component. The system is organized conceptually into independent responsibilities: game-session/state control, snake movement, food generation, scoring, collision detection, difficulty configuration and local high-score persistence. Core game logic remains separable from SFML rendering to support unit testing. Section 4 shows the use-case structure; there is no larger system it plugs into.

2.2 Product Functions

- Display a start screen and let the player start a new game session.
- Move the snake continuously and change its direction on keyboard input.
- Spawn food at random unoccupied grid positions and grow the snake on consumption.
- Detect wall and self-collision and transition to a game-over state.
- Track and display the current score during play.
- Pause and resume an active session.
- Offer selectable difficulty levels that change snake speed.
- Persist and reload the all-time high score from a local file.
- Allow the player to restart directly from the game-over state.

2.3 User Classes and Characteristics

User Class	Characteristics
Player	Non-technical, casual end user; interacts only through keyboard and on-screen prompts; expects immediate, responsive feedback (< 1 frame input lag).
Course Evaluator	Technical reviewer; inspects source code, build process, Git history, test evidence; does not need a distinct in-app role, but is treated as the key documentation stakeholder.

2.4 Operating Environment

- Desktop OS: Windows 10/11 or Linux (Ubuntu 22.04+), 64-bit.
- Runtime: Native compiled C++17 executable, no interpreter required.
- Graphics: SFML 2.6 (requires OpenGL-capable GPU/driver, minimum 1024×768 display).
- Build environment: CMake 3.16+, GCC/Clang/MSVC with C++17 support.

2.5 Design and Implementation Constraints

- Must be implemented in C++17, using SFML for rendering, audio and input — no alternative game engine.
- Business/game logic must be decoupled from SFML-specific rendering calls to remain unit-testable without a display context.
- Only one active game session shall exist per running application instance.
- High-score data must be stored in a local, human-readable file — no external database or network service.
- Target frame rate: 60 FPS for rendering; game-tick (snake movement) rate is decoupled and scales with difficulty.

2.6 Assumptions and Dependencies

- Assumes the evaluator's/player's machine has a graphical display; the game is not designed for headless/CI execution beyond unit tests of the logic layer.
- Depends on the SFML 2.6 library being available at build time (via package manager or vendored submodule).
- Assumes a single local player per session; no concurrency or multi-user handling is required.
- Version 1.0 does not depend on network connectivity or cloud services.

3. External Interface Requirements

3.1 User Interfaces

The game exposes four screens: Start Screen (title, difficulty selector, high score, 'Press Enter to Play'), Gameplay Screen (grid, snake, food, live score HUD), Pause Overlay ('Paused — Press P to Resume') and Game Over Screen (final score, new-high-score indicator, Restart). All screens use a consistent color theme and font, and all interactive prompts are keyboard-driven (Arrow keys / WASD for movement, P to pause, Enter/R/Esc for screen transitions). UI behavior shall be clear enough that a first-time player can understand the current game state and available action without consulting source code or external instructions.

3.2 Hardware Interfaces

The application requires standard desktop input/output hardware: a display and a keyboard. No specialized hardware interface is required.

3.3 Software Interfaces

Component	Version	Purpose
SFML — Graphics module	2.6.x	Window creation, sprite/shape rendering, HUD/menu text
SFML — Window/System module	2.6.x	Keyboard input polling, game clock/delta-time
SFML — Audio module (optional)	2.6.x	Sound effects for food consumption / game over
Local filesystem	OS-native	Read/write the high-score record file

3.4 Communications Interfaces

None. SnakeArcade is a fully offline, single-machine application; it performs no network communication, and there are no communication protocols, ports or data-transfer rates to specify.

4. Analysis Models

4.1 Use Case Diagram

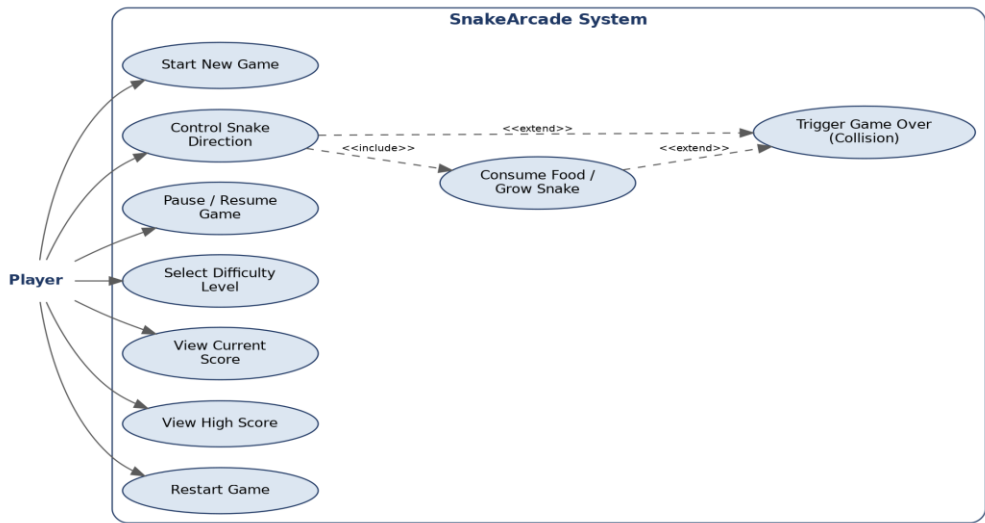


Figure 1 — Use case diagram for the Player actor interacting with SnakeArcade.

4.2 Domain / Class-Relationship Diagram

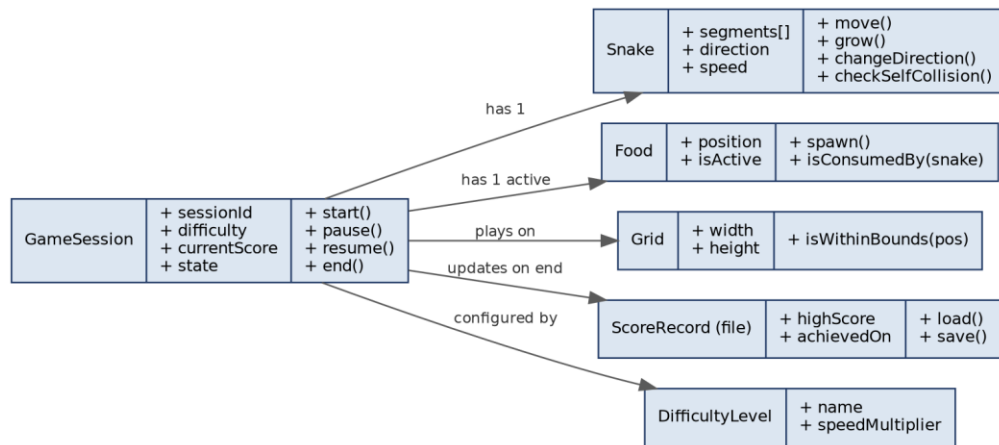


Figure 2 — Core domain entities: GameSession, Snake, Food, Grid, DifficultyLevel and the persisted ScoreRecord.

4.3 Core Game-State Model

The system operates using four primary states: START, PLAYING, PAUSED and GAME_OVER. Starting a game transitions START → PLAYING. Pausing transitions PLAYING → PAUSED. Resuming transitions PAUSED → PLAYING. A game-ending collision transitions PLAYING → GAME_OVER. Restarting from Game-Over creates a new PLAYING session directly, without requiring a separate return-to-start step.

5. System Features

5.1 Game Session Management

Description and Priority: Controls the transition between start, active gameplay and game-over states and supports direct restart. Priority: High.

Stimulus/Response: Player launches the application → start screen is displayed. Player selects Start → a new session is initialized and gameplay begins. Player selects Restart from Game-Over → the session resets and gameplay begins again.

ID	Requirement	Priority	Verification
FR-GSM-001	The system shall display a start screen when the application launches.	High	System test
FR-GSM-002	The system shall initialize a new session (snake, score, grid and food set to default state) when the player starts a game.	High	Functional test
FR-GSM-003	The system shall transition to a game-over state when the active session ends.	High	System test
FR-GSM-004	The system shall allow the player to restart directly from the game-over screen, without a separate return-to-start-screen step.	High	System test

5.2 Snake Movement and Input

Description and Priority: Handles keyboard direction input and continuous movement on the bounded grid. Priority: High.

Stimulus/Response: Player presses a supported direction key → the requested direction is registered (subject to the 'no 180° reversal' rule) → on the next game update the snake advances one grid cell in that direction.

ID	Requirement	Priority	Verification
FR-MOV-001	The system shall accept Up/Down/Left/Right (Arrow or WASD) directional input from the player.	High	Functional test
FR-MOV-002	The system shall move the snake forward by exactly one grid cell on every game update.	High	Unit test
FR-MOV-003	The system shall calculate movement using elapsed time so behavior is independent of rendering frame rate.	High	Unit + system test
FR-MOV-004	The system shall reject a direction change that would reverse the snake directly onto its own neck (180° reversal).	High	Unit test

5.3 Food, Growth and Scoring

Description and Priority: Generates food, detects consumption and updates snake length and score. Priority: High.

Stimulus/Response: Snake head enters the food's grid cell → food consumption is detected, score increases, and the snake grows by one segment.

ID	Requirement	Priority	Verification
FR-FGS-001	The system shall spawn food at a random, valid grid cell not currently occupied by the snake.	High	Unit test
FR-FGS-002	The system shall detect when the snake's head occupies the same cell as the active food item.	High	Unit test
FR-FGS-003	The system shall increase the snake's length by one segment when food is consumed.	High	Unit test
FR-FGS-004	The system shall increase the current score by a fixed point value when food is consumed.	High	Unit test
FR-FGS-005	The system shall update and display the current score on the Gameplay screen within the same update it changes.	High	Unit + system test

5.4 Collision and Game Over

Description and Priority: Detects collisions with the play-area boundary and the snake's own body. Priority: High.

Stimulus/Response: Snake head reaches a boundary, or overlaps its own body → a terminal collision is detected and the game enters the game-over state.

ID	Requirement	Priority	Verification
FR-CGS-001	The system shall detect collision between the snake's head and the play-area boundary.	High	Unit test
FR-CGS-002	The system shall detect collision between the snake's head and any other segment of its own body.	High	Unit test
FR-CGS-003	The system shall stop active gameplay updates immediately upon detecting a terminal collision.	High	System test
FR-CGS-004	The system shall display the final score on the game-over screen.	Medium	System test

5.5 Pause and Resume

Description and Priority: Allows an active game session to be paused and resumed without resetting the current state.

Priority: Medium.

Stimulus/Response: Player issues the pause command during active gameplay → movement and gameplay updates stop. Player issues the resume command → the same game state continues.

ID	Requirement	Priority	Verification
FR-PRS-001	The system shall allow the player to pause an active game session.	Medium	Functional test
FR-PRS-002	The system shall prevent gameplay updates (movement, timers) while the session is paused.	High	System test
FR-PRS-003	The system shall resume the same game state — snake position, score and food location unchanged — when the player resumes.	Medium	System test

5.6 Difficulty Selection

Description and Priority: Provides three selectable difficulty levels that alter snake movement speed. Priority: High.

Stimulus/Response: Player selects Easy, Medium or Hard → the corresponding movement speed is configured for the next game session and used for its full duration.

ID	Requirement	Priority	Verification
FR-DIF-001	The system shall provide Easy, Medium and Hard difficulty levels selectable before starting a session.	High	Functional test
FR-DIF-002	The system shall associate each difficulty level with a distinct snake movement speed.	High	Unit + system test

ID	Requirement	Priority	Verification
FR-DIF-003	The system shall apply the selected difficulty's speed for the entire duration of the session.	High	System test

5.7 High Score Persistence

Description and Priority: Stores the highest achieved score locally and restores it on subsequent application runs. Priority: High.

Stimulus/Response: Application starts → saved high score is read if the persistence file exists and is valid. Player completes a session → score is compared with, and if greater, replaces, the saved high score.

ID	Requirement	Priority	Verification
FR-HSC-001	The system shall load the persisted high score at application startup if the persistence file exists and is valid.	High	Integration test
FR-HSC-002	The system shall display the current high score to the player.	Medium	System test
FR-HSC-003	The system shall compare a completed session's score against the stored high score when the session ends.	High	Unit test
FR-HSC-004	The system shall update and persist the high score when a completed session's score exceeds it.	High	Integration test
FR-HSC-005	The system shall handle a missing or corrupted high-score file by defaulting to zero, without crashing.	High	Negative test

6. Other Nonfunctional Requirements

Non-functional requirements below are kept deliberately lean: each ID'd item is individually measurable and warrants its own verification activity. Broader design-quality expectations that are best verified through architecture/code review rather than a dedicated test case are captured separately in Section 6.4 as descriptive Quality Attributes.

6.1 Performance Requirements

ID	Requirement	Verification
NFR-PERF-001	Gameplay movement shall be calculated using elapsed time so behavior is independent of rendering frame rate.	Timing / system test
NFR-PERF-002	Valid directional input shall be processed within one game update after being received.	System test
NFR-PERF-003	High-score load and save operations shall complete without a perceptible pause in gameplay or screen transitions.	System test

6.2 Safety / Reliability Requirements

ID	Requirement	Verification
NFR-REL-001	The application shall not intentionally modify any file other than its designated high-score persistence file.	Code review
NFR-REL-002	Malformed persistence data shall not cause uncontrolled program termination.	Negative test
NFR-REL-003	Invalid or unsupported keyboard input shall not cause undefined game-state transitions.	Negative test

6.3 Usability Requirements

ID	Requirement	Verification
NFR-USE-001	A new game shall be startable using no more than 2 keypresses from application launch.	Usability walkthrough
NFR-USE-002	The Gameplay screen shall visibly display the current score at all times during active play.	UI test

6.4 Software Quality Attributes

These attributes describe design-level expectations verified primarily through architecture and code review rather than a dedicated per-item test case.

Attribute	Requirement	Verification
Modularity	Input handling, game logic, rendering and persistence shall be implemented as separated modules/classes.	Architecture / code review
Maintainability	Core gameplay rules shall be implemented independently of SFML-specific rendering calls.	Code review
Testability	Movement, collision and scoring logic shall be unit-testable without opening a graphical window.	Unit-test review
Portability	The application shall build and run on the team's documented target OS/toolchain without platform-specific game logic.	Build/release test
Extensibility	Difficulty configuration shall be data-driven so new levels can be added without changing core movement/collision logic.	Design review

6.5 Security Requirements

ID	Requirement	Verification
SEC-001	Persisted high-score values shall be validated before use; invalid values shall not be applied.	Negative test
SEC-002	The system shall not execute or interpret any content read from the high-score persistence file (treated strictly as data, not code).	Code review
SEC-003	The application shall recover safely (default to zero) if the high-score file cannot be read or written.	Negative / security test

7. Business / Domain Rules

- A game session shall have exactly one active snake state and one current score at a time.
- Food shall be positioned within the bounded play area and shall not spawn on a cell occupied by the snake.
- Consuming food shall increase both snake length and score by their defined fixed increments.
- Collision with the wall or the snake's own body shall terminate the active session.
- Only an active (PLAYING) session shall respond to movement updates; a paused session shall not.
- The selected difficulty shall determine the movement-speed configuration for the entire session.
- The stored all-time high score shall never decrease as a result of a normal game session, and shall only be replaced by a strictly greater completed-session score.

8. Data / Field Requirements

8.1 High-Score Record (persisted file)

SnakeArcade does not require a relational database. The only persistent data identified in the Synopsis is the all-time high score.

Field	Type	Description	Validation
high_score	Integer	Greatest final score achieved across all sessions	Non-negative integer; reject malformed values

8.2 Game Session Data (in-memory, not persisted)

Field	Type	Description
snake_positions	Collection of grid coordinates	Ordered positions representing the snake's body, head first
direction	Enum	Current valid movement direction
food_position	Grid coordinate	Current valid food location
score	Integer	Current session score
game_state	Enum	START / PLAYING / PAUSED / GAME OVER

Appendix A: Glossary

Term	Definition
Grid	The fixed-size 2D playing field, divided into discrete cells, on which the snake and food are positioned.
Tick	One discrete game-logic update step, at which the snake advances one cell; its interval is controlled by difficulty.
Segment	One unit of the snake's body occupying a single grid cell.
Session	One complete playthrough from Start Screen to Game-Over.
High Score	The greatest final score achieved across all sessions, persisted locally.
Game State	Current application gameplay mode: Start, Playing, Paused or Game-Over.

Appendix B: Requirement Group Traceability Summary

This summary table groups requirements by feature and assigns primary implementation ownership, consistent with the Synopsis's team-ownership matrix. It complements the detailed per-requirement matrix in Appendix C.

Requirement Group	Feature / Component	Primary Owner	Planned Verification
FR-GSM-001 to FR-GSM-004	Game session management	Rohil	Functional & system tests
FR-MOV-001 to FR-MOV-004	Snake movement and input	Rohil	Unit & system tests
FR-FGS-001 to FR-FGS-005	Food, growth and scoring	Sai Harshith	Unit & system tests
FR-CGS-001 to FR-CGS-004	Collision and game over	Sai Harshith	Unit & system tests
FR-PRS-001 to FR-PRS-003	Pause and resume	Sarayu	Functional & system tests
FR-DIF-001 to FR-DIF-003	Difficulty selection	Sarayu	Functional & system tests
FR-HSC-001 to FR-HSC-005	High-score persistence	Sarayu	Integration & negative tests
NFR-PERF-*, NFR-REL-*, NFR-USE-*	Performance, reliability, usability	Shared	Timing / negative / UI tests
Quality Attributes (6.4), SEC-*	Maintainability, testability, security	Shared	Code/repository review, security tests

Appendix C: Requirement Traceability Matrix (Initial Baseline)

This matrix will be completed progressively as architecture, design, code and tests are produced in later deliverables, following the chain Requirement ID → Architecture Reference → Design Reference → Code File Reference → Test Case ID → System Test ID. Values shown below are the planned references based on current design intent and will be finalized in the Design and Implementation deliverables.

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-GSM-002	Initialize new session on start	UI-01	Class: GameController	game_session.cpp	UT-SESSION-01	ST-SESSION-01
FR-MOV-001	Move snake one cell/tick	GAME-LOOP-01	Class: Snake	snake.cpp	UT-SNAKE-01	ST-PLAY-01
FR-MOV-003	Block 180° reversal	GAME-LOOP-01	Class: Snake	snake.cpp	UT-SNAKE-02	ST-PLAY-02
FR-FGS-001	Spawn food on empty cell	LOGIC-02	Class: Food	food.cpp	UT-FOOD-01	ST-PLAY-03
FR-CGS-001	Wall collision ends game	LOGIC-03	Class: CollisionDetector	collision.cpp	UT-COLL-01	ST-COLL-01

Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
FR-PRS-001	Pause active session	STATE-01	Class: GameStateManager	game_state.cpp	UT-STATE-01	ST-STATE-01
FR-HSC-001	Load persisted high score	PERSIST-01	Class: ScoreRecord	score_record.cpp	UT-SCORE-01	ST-SCORE-01
FR-HSC-005	Handle missing/corrupt score file	PERSIST-01	Class: ScoreRecord	score_record.cpp	UT-SCORE-02	ST-SEC-01
SEC-001	Validate persisted file contents	PERSIST-01	Class: ScoreRecord	score_record.cpp	UT-SEC-01	ST-SEC-01

Appendix D: Initial Validation Specification

At the requirements stage, this appendix is planning-focused rather than a final test report. It explains how functional, security and non-functional requirements will eventually be validated, following the Level 3 guideline to plan functional, security and non-functional validation, including negative/security testing.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Start game, control snake, pause/resume, restart
Collision	Unit + system tests	Wall collision and self-collision end the session
Scoring	Unit + system tests	Food consumption increases score and length
Difficulty	Functional + system tests	Each difficulty applies a distinct, consistent speed
Performance	Timing / system tests	Elapsed-time movement under different difficulties
Usability	Task-based walkthrough	First-time player starts, plays, pauses and restarts unaided
Reliability	Negative / failure tests	Missing/corrupted high-score file does not crash the app
Security	Input validation + code review	Malformed persistence data and restricted file writes

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering
UE24CS341A – Software Engineering | Jackfruit Mini-Project

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Project: SnakeArcade

Item	Details
Project	SnakeArcade (Project ID: SG-2026-01, Team 1 / G-01)
Version	1.0
Authors	Rohil Thimmanahalli Girish, Sai Harshith R, Sarayu Mayakonda
Date	September 2026
Status	Final — Lab 3 Submission

Revision History

Version	Date	Author	Change Summary
1.0	September 2026	Team 1	Initial Architecture and Design Specification, derived from the approved Synopsis and SRS v1.0.

Approvals

Role	Name	Signature / Date
Team Member	Rohil Thimmanahalli Girish	
Team Member	Sai Harshith R	
Team Member	Sarayu Mayakonda	
Course Instructor		

Team

Name	USN	SRN	Primary Component Ownership
Rohil Thimmanahalli Girish	PES1UG24CS384	PES1202402234	Game Controller, Input Handler, Game Logic (session, movement)
Sai Harshith R	PES1UG24CS398	PES1202400250	Collision Manager, Food & Score Manager
Sarayu Mayakonda	PES1UG24CS926	PES1202404807	Persistence Manager (difficulty, pause/resume, high score)

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of SnakeArcade, a single-player desktop Snake game built in C++ using the SFML graphics library. It records the architectural-pattern selection, component decomposition, key interaction flows, security architecture and detailed design (sequence diagrams, API/interface design and error handling) that will guide implementation.

1.2 Scope

Covers the internal software architecture and design of SnakeArcade: game-session management, snake movement and input handling, food/growth/scoring, collision detection, pause/resume, difficulty selection and local high-score persistence, as defined in the approved SRS v1.0. It does not cover hardware, operating-system or SFML-internal design, which are treated as external dependencies.

1.3 Audience

Developers (all three team members), the course instructor/evaluator assessing architectural quality and requirement traceability, and any future maintainer of the codebase.

1.4 Definitions

Term	Definition
MVC	Model–View–Controller architectural pattern.
Component	An independently identifiable, replaceable unit of software with a well-defined responsibility and interface.
Lifeline	In a UML sequence diagram, the vertical dashed line representing an object's existence over time.
SFML	Simple and Fast Multimedia Library — the external C++ graphics/window/input library used for rendering.
ARCH Ref	Architecture reference code used in the SRS traceability matrix (e.g. GAME-LOOP-01, PERSIST-01) to link a requirement to an architectural component.

2. Document Overview

2.1 How to use this document

Section 3 (Architecture) explains the high-level structure of the system: the goals and constraints driving the design, the chosen architectural pattern and the alternatives considered, the component model, and the security architecture. Section 4 (Design) provides the detailed design used for implementation: UML sequence diagrams for the two most important flows, the internal API/interface design between components, and error-handling conventions. Appendices provide a glossary, references and the tools used.

2.2 Related Documents

- SnakeArcade Software Requirements Specification (SRS), v1.0.
- SnakeArcade Synopsis / Project Proposal, v1.0.
- SnakeArcade Software Test Plan (STP), v1.0 (companion Lab 3 deliverable).

3. Architecture

3.1 Goals & Constraints

Goals: modularity (input, game logic, rendering and persistence must remain separately identifiable), testability (movement, collision and scoring should be testable independently of graphical presentation where practical), maintainability (a change to one gameplay module should minimize unnecessary changes elsewhere), and reliability (invalid input and missing/corrupted persistence data should not crash the application), consistent with the SRS Quality Attributes and NFRs.

Constraints: C++17 with SFML only (no alternative game engine); no external database or network service; the only persisted data is the local high-score file; the team has a single semester and roughly 1–2 days/week of available time, so the architecture must stay proportionate to a small, well-understood scope.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Player	Responsive controls, a game that never crashes, and a correctly remembered high score.
Course Instructor / Evaluator	Clear architectural rationale, traceability from requirements to components, and testable, maintainable code.
Development Team	A structure simple enough to implement and test within limited weekly time, with clear per-member component ownership.

3.3 Component (UML) Diagram

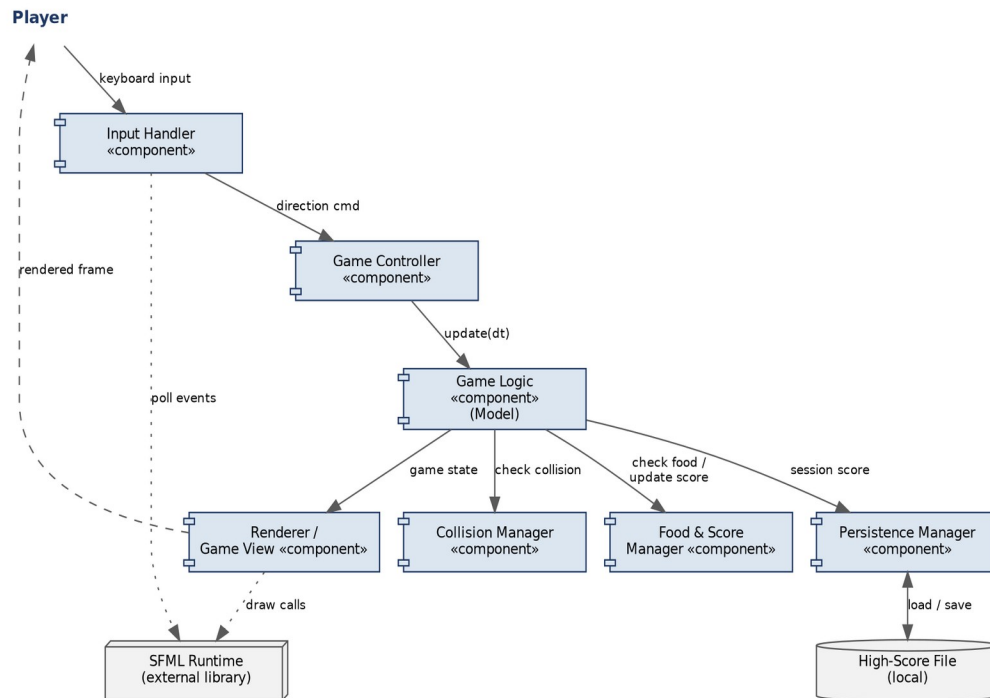


Figure 1 — UML component diagram for SnakeArcade, showing the Player actor, internal components, and the external SFML Runtime and High-Score File dependencies.

3.4 Component Descriptions

Component	Layer	Responsibility	Key Operations	Depends On
Input Handler	Controller (input)	Polls SFML keyboard events and translates them into logical direction commands.	handleInput(), getRequestedDirection()	SFML Runtime
Game Controller	Controller	Coordinates game-state transitions (Start, Playing, Paused, Game-Over) and drives the game loop.	startSession(), pause(), resume(), update(dt)	Input Handler, Game Logic
Game Logic	Model	Owns snake, food, score and difficulty state; enforces core movement, growth and scoring rules.	update(dt), applyDirection(), getState()	Collision Manager, Food & Score Manager
Collision Manager	Model (sub-component)	Detects wall collision and self-collision against the current snake state.	checkWallCollision(), checkSelfCollision()	Game Logic
Food & Score Manager	Model (sub-component)	Spawns food, detects consumption, and updates score and snake length.	spawnFood(), isConsumed(), applyScore()	Game Logic
Renderer / Game View	View	SFML-based rendering of the Start, Gameplay, Pause and Game-Over screens.	render(state)	Game Logic, SFML Runtime
Persistence Manager	Persistence	Loads, validates and saves the local high-score record.	loadHighScore(), saveHighScore(), validate()	High-Score File
SFML Runtime	External Library	Third-party window, graphics, input-polling and timing primitives.	—	—
High-Score File	External Data Store	Local file holding the persisted all-time high score.	—	—

3.5 Chosen Architecture Pattern and Rationale

The proposed architectural pattern for SnakeArcade is Model–View–Controller (MVC), implemented with modular game components. MVC is suitable because the game naturally separates the player/input control flow, the game state and rules, and the graphical presentation.

The pattern is applied as follows:

- Model: game state and core rules — snake state, food, score, collision results, difficulty and high-score data (Game Logic, Collision Manager, Food & Score Manager).
- View: SFML-based rendering of the start screen, gameplay, pause state, score and game-over state (Renderer / Game View).
- Controller: input handling and game-state coordination — receives keyboard events, normalizes player actions and invokes the appropriate Model operations (Input Handler, Game Controller).
- Persistence: a separate infrastructure component supporting the Model, responsible only for reading and writing the local high score (Persistence Manager).

Why MVC fits SnakeArcade

- The player interacts through keyboard input, which can be handled independently from the rules that update the game state.
- The rendering layer can display the current state without owning gameplay rules, reducing coupling between SFML drawing code and game logic.
- Core gameplay can be unit-tested without requiring the complete graphical runtime, matching the SRS testability requirement.
- Difficulty and gameplay rules can be extended without rewriting the rendering/input layer.
- The local high-score file is isolated behind a persistence component, making file-validation and failure handling easier to test.
- The pattern maps cleanly to the logical architecture already described in the SRS.

Alternatives Considered

Alternative	Description	Reason Rejected / Accepted
Monolithic single-file design	All input, game-rule and rendering code in one undivided structure.	Rejected — violates the SRS modularity and testability requirements; core logic could not be unit-tested independently of SFML.
Entity-Component-System (ECS)	Data-oriented pattern common in larger games with many diverse game-object types.	Rejected — disproportionate complexity for a single-snake, single-food game; would cost more implementation time than the team's schedule allows.
Model–View–Controller (chosen)	Separates input/control, game state/rules, and rendering into distinct components.	Selected — matches the natural structure of the game, is well understood by the team, and directly supports the SRS's modularity and testability NFRs.

3.6 Technology Stack & Data Stores

C++17, SFML 2.6 (Graphics, Window/System and optional Audio modules), CMake 3.16+ build system. Data store: a single local, human-readable high-score file (see SRS Section 8.1); no relational database, no network-based data store.

3.7 Risks & Mitigations

Risk	Mitigation
High-score file becomes corrupted or unreadable.	Persistence Manager validates format/range on load and defaults to zero without crashing (SRS FR-HSC-005, NFR-REL-002).
Movement/collision logic becomes tightly coupled to SFML rendering, blocking unit testing.	Game Logic, Collision Manager and Food & Score Manager expose interfaces with no direct SFML dependency (SRS NFR-MNT / Quality Attributes).
Limited weekly team time (1–2 days/week) causes scope overrun.	Architecture kept intentionally small — 7 components, no network/auth/database layer — matching the trimmed SRS scope.

Risk	Mitigation
Uneven component complexity across the 3 owners.	Component ownership assigned per Section “Team” table, grouping related components (e.g., Collision + Food/Score) under one owner.

3.8 Traceability to Requirements

The Level 3 process expects requirements to trace through architecture, design, code and tests. The table below links key SRS requirements to the architecture references used in the SRS Appendix C traceability matrix and to the owning component defined in Section 3.4.

Req ID	Requirement (brief)	Architecture Ref	Owning Component
FR-GSM-002	Initialize new session on start	UI-01	Game Controller
FR-MOV-001 / 003	Move snake one cell/tick; block 180° reversal	GAME-LOOP-01	Game Logic
FR-FGS-001–005	Spawn food, detect consumption, grow, update score	LOGIC-02	Food & Score Manager
FR-CGS-001–004	Detect wall/self collision, end session	LOGIC-03	Collision Manager
FR-PRS-001–003	Pause / resume session	STATE-01	Game Controller
FR-DIF-001–003	Offer and apply difficulty levels	STATE-01	Game Controller / Game Logic
FR-HSC-001–005	Load, compare, persist and recover high score	PERSIST-01	Persistence Manager
NFR-PERF-001	Elapsed-time based movement	GAME-LOOP-01	Game Logic
SEC-001–003	Validate, isolate and safely recover persisted data	PERSIST-01	Persistence Manager

3.9 Security Architecture

SnakeArcade has no network-facing service, authentication subsystem or external database. Therefore, the main security boundary is the local persistence mechanism and the validation of data crossing that boundary.

- Persistence writes are restricted to the designated high-score file location (Persistence Manager only).
- Loaded high-score data is validated for format and numeric range before use.
- The application treats high-score file contents strictly as data and never executes or interprets them as commands.
- No unnecessary personal or sensitive user information is stored.
- Missing, unreadable or corrupted high-score data is handled without preventing a new game from starting.

Threat	Mitigation
Malicious or corrupted high-score file content	Validate format and numeric range before use; default to zero on failure (SEC-001, FR-HSC-005).
Unauthorized or unintended file writes elsewhere on disk	Persistence Manager is the only component with file-write access, restricted to one designated file path.
Arbitrary file content executed as code	File content is treated strictly as data; the application never evaluates or executes it (SEC-002).
Sensitive data exposure	No personal or sensitive user data is stored; only a numeric high score is persisted (SEC-003).
Application crash from malformed input or file data	Defensive validation in Persistence Manager and Input Handler prevents uncontrolled crashes (NFR-REL-002/003).

4. Design

4.1 Design Overview

SnakeArcade is designed with layered, MVC-aligned modules for separation of concerns and easier maintenance. The Game Controller coordinates the game loop and state transitions; the Game Logic component and its two sub-components (Collision Manager, Food & Score Manager) own all gameplay rules; the Renderer draws the current state via SFML; and the Persistence Manager isolates all file I/O behind a small, testable interface.

4.2 UML Activity Diagram

The activity diagram below shows the core per-tick gameplay loop — the control flow that runs once every game update while a session is PLAYING, covering input, movement, collision checks and food consumption.

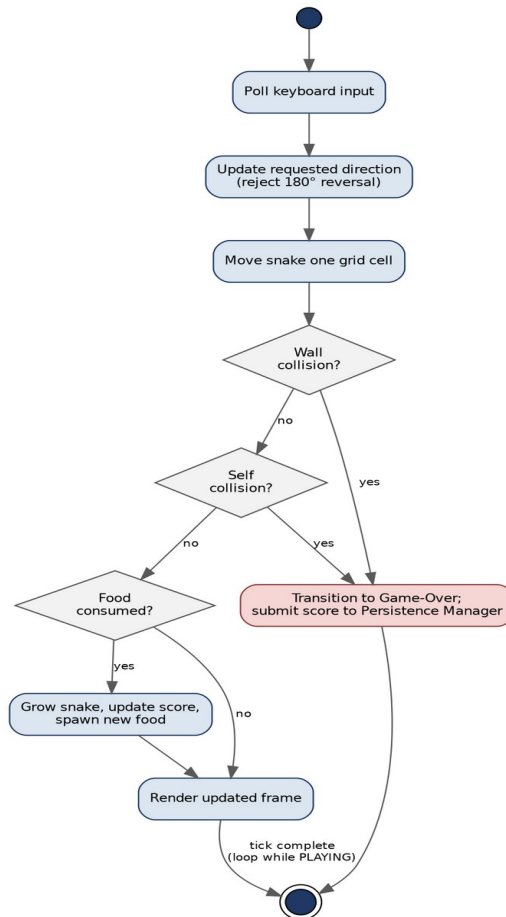


Figure 2 — Activity diagram: one game-tick cycle (poll input → move → check collisions → check food → render).

4.2a Deployment Diagram

Not applicable. SnakeArcade is a single-machine desktop binary with no distributed runtime nodes, containers or network-deployed services — the entire application (Input Handler, Game Controller, Game Logic, Renderer and Persistence Manager) runs in one process on the player's machine, writing only to a local file. A deployment diagram would not convey additional information beyond Section 3.6 Technology Stack.

4.3 UML Sequence Diagrams

Two sequence diagrams are provided, covering the two flows most central to SnakeArcade's gameplay: (1) a single movement tick that may consume food, and (2) the transition from a terminal collision through game-over to a persisted high-score update.

Figure 3 — Sequence: Move and Consume Food

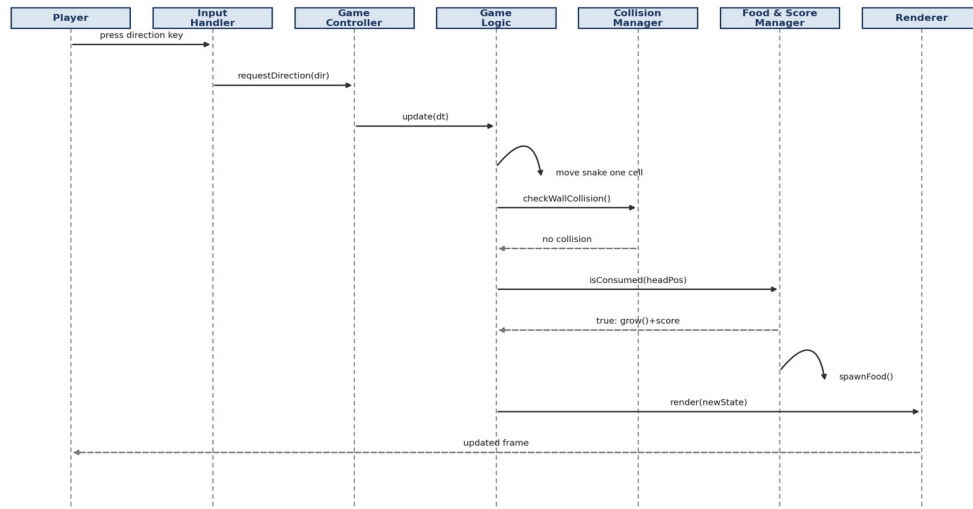


Figure 3 — Sequence diagram: Move and Consume Food.

Figure 4 — Sequence: Collision, Game Over and High-Score Update

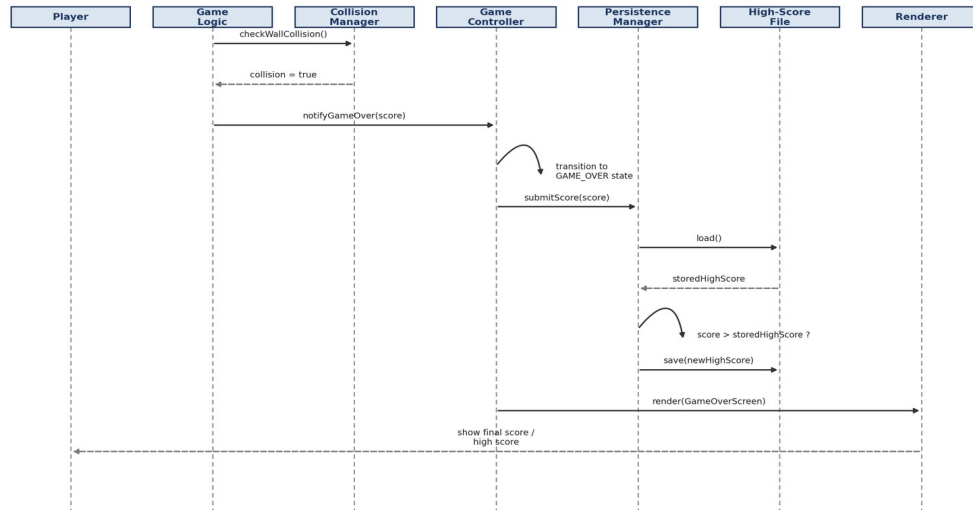


Figure 4 — Sequence diagram: Collision, Game Over and High-Score Update.

4.4 API Design

SnakeArcade is a local desktop application with no network-facing REST API. “API design” here refers to the internal interface each component exposes to its collaborators, which plays the same role a REST endpoint would in a networked system. Interfaces for at least two components are defined below.

Game Logic Interface

Operation	Parameters	Returns	Notes
update(dt)	dt: elapsed time (seconds)	void	Advances the snake by one cell when the tick interval elapses; invokes Collision Manager and Food & Score Manager.
applyDirection(dir)	dir: Up/Down/Left/Right	bool (accepted)	Rejects a direction that would reverse the snake onto its own neck (FR-MOV-004).
getState()	—	GameState (snake, food, score)	Read-only snapshot consumed by the Renderer.

Persistence Manager Interface

Operation	Parameters	Returns	Notes
loadHighScore()	—	int (default 0 on failure)	Validates file contents before returning a value (SEC-001, FR-HSC-005).
saveHighScore(score)	score: int	bool (success)	Called only when score exceeds the currently stored high score (FR-HSC-004).
validate(rawData)	rawData: file contents	bool	Checks format and numeric range; used internally by loadHighScore().

API Contract Rules

- Define the parameters and return type/schema for every operation (as in the tables above).
- Define the failure/edge-case behavior for each operation (e.g., loadHighScore() returns 0 rather than throwing on a missing file).
- Define which operations mutate state (applyDirection(), saveHighScore()) versus which are read-only (getState(), loadHighScore()).
- Keep interface names and signatures consistent between this document, the code and the SRS traceability matrix (Appendix C).
- Do not change a component's interface signature silently once another component depends on it — update this document and notify the owning members first.

4.5 Error Handling, Logging & Monitoring

- Standardized, human-readable messages for the small set of recoverable errors (e.g., “high score file unreadable, starting from 0”).
- No sensitive information is written to any log or console output.
- Invalid keyboard input and out-of-range file data are handled defensively and never propagate as unhandled exceptions (NFR-REL-002/003).
- For a project of this scope, monitoring is limited to development-time diagnostics (console warnings during testing); no production monitoring service is required.

4.6 UX Design

The UI favors clarity over decoration: a fixed-size play grid, a persistently visible score, clear state-appropriate prompts (“Press Enter to Play”, “Paused — Press P to Resume”, “Game Over — Press R to Restart”), and consistent font/color use across all four screens, satisfying the SRS usability requirements (NFR-USE-001/002).

4.7 Open Issues & Next Steps

- Finalize exact grid dimensions and tick-interval values per difficulty level during implementation.
- Confirm the on-disk format (plain text vs. simple key–value) for the high-score file with the whole team before Persistence Manager implementation begins.
- Future enhancement (out of current scope): optional sound effects via the SFML Audio module.

5. Appendices

5.1 Glossary

See Section 1.4 Definitions above, and the Glossary in the SRS (Appendix A) for gameplay terms (Grid, Tick, Segment, Session, High Score, Game State).

5.2 References

- SnakeArcade Software Requirements Specification (SRS), v1.0.
- PES University Jackfruit Mini-Project — Software Engineering Project Guidelines, 2026 (UE24CS341A).

- PES University SE Level-3 Student Submission Guide & Coffee Kiosk Exemplar, 2026.

5.3 Tools

Graphviz (component diagram), a custom UML-style sequence-diagram renderer (sequence diagrams), CMake (build), SFML 2.6 (runtime library).

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
UE24CS341A – Software Engineering | Jackfruit Mini-Project

SOFTWARE TEST PLAN (STP)

Project: SnakeArcade

Item	Details
Project	SnakeArcade (Project ID: SG-2026-01, Team 1 / G-01)
Version	1.0
Authors	Rohil Thimmanahalli Girish, Sai Harshith R, Sarayu Mayakonda (QA responsibilities shared by all members)
Date	September 2026
Status	Final — Lab 3 Submission

1. Introduction

Purpose: this document defines the test plan for SnakeArcade v1.0. It outlines objectives, scope, strategy, resources, schedule and responsibilities for testing, following the approved SRS v1.0 and SAD v1.0.

Scope: testing covers game-session management, snake movement and input, food/growth/scoring, collision detection, pause/resume, difficulty selection and local high-score persistence. Hardware, the operating system, and the SFML library's internal correctness are excluded.

References: SnakeArcade SRS v1.0; SnakeArcade Software Architecture and Design Specification (SAD) v1.0; SnakeArcade Synopsis v1.0.

Definitions: FR (Functional Requirement), NFR (Non-Functional Requirement), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), TC (Test Case).

2. Test Items

- Game Session Management module
- Snake Movement and Input module
- Food, Growth and Scoring module
- Collision and Game Over module
- Pause and Resume module
- Difficulty Selection module
- High-Score Persistence module

3. Features to be Tested

Features mapped to SRS requirement IDs:

SRS Requirement ID	Feature
FR-GSM-001 to FR-GSM-004	Start screen, session init, game-over transition, restart
FR-MOV-001 to FR-MOV-004	Directional input, elapsed-time movement, 180° reversal block
FR-FGS-001 to FR-FGS-005	Food spawning, consumption, growth, scoring
FR-CGS-001 to FR-CGS-004	Wall/self collision detection, game-over transition
FR-PRS-001 to FR-PRS-003	Pause, resume, state preservation
FR-DIF-001 to FR-DIF-003	Difficulty selection and speed configuration
FR-HSC-001 to FR-HSC-005	High-score load, compare, persist, corrupted-file recovery

SRS Requirement ID	Feature
NFR-PERF-001 to NFR-PERF-003	Elapsed-time movement, input responsiveness, I/O responsiveness
NFR-REL-001 to NFR-REL-003	File-write restriction, malformed-data handling, invalid-input handling
NFR-USE-001 to NFR-USE-002	Startup keypress count, visible score display
SEC-001 to SEC-003	Persisted-data validation, no code execution from file, safe recovery

4. Features Not to be Tested

- SFML library internals (rendering engine, window management) — assumed correct as a third-party dependency.
- Operating system keyboard driver behavior.
- Build-toolchain (CMake/compiler) correctness.
- Any feature outside the approved SRS scope (e.g., multiplayer, settings screen, network play) — explicitly out of scope per SRS Section 7 Scope.

5. Test Approach / Strategy

Levels

- Unit tests (module-level: movement, collision, scoring, persistence logic)
- Integration tests (e.g., Game Logic ↔ Persistence Manager; Game Controller ↔ Game Logic)
- System tests (end-to-end gameplay, from start screen through game-over)

Types

- Functional testing (core gameplay features)
- Negative testing (invalid input, corrupted persistence data)
- Performance/timing testing (elapsed-time movement, input responsiveness)
- Usability testing (startup flow, on-screen clarity)

Entry Criteria

A stable, buildable game session (start → play → game-over) is available; test data (a small set of high-score file fixtures, including a deliberately corrupted one) is prepared; the local build/test environment is set up on at least one team member's machine.

Exit Criteria

All planned test cases (Section 13.1) executed; all High-priority functional requirements pass; no open defect blocks core gameplay (start, move, collide, restart); results recorded with Actual Result and Pass/Fail.

5.1 Security Validation

- Validate high-score file handling: malformed, empty and missing files must not crash the application and must default to a high score of zero (SEC-001, FR-HSC-005).
- Confirm persisted file contents are never executed or interpreted as code (SEC-002) — verified by code review of the Persistence Manager.
- Confirm the Persistence Manager writes only to its single designated high-score file path and no other location (Section 3.9 of the SAD).
- Confirm no personally identifiable or sensitive information is stored in the persisted file (SEC-003).
- Fuzz-style negative testing: feed the high-score loader a range of invalid inputs (non-numeric text, negative numbers, truncated files) and confirm graceful fallback to defaults in every case.

6. Test Environment

Hardware: standard desktop/laptop with keyboard and a 1024×768-or-larger display.

Software: SnakeArcade build (C++17, SFML 2.6), built via CMake 3.16+.

Tools: GoogleTest (gtest) for unit/integration tests; manual play-testing for system/usability tests; GitHub Actions (or manual local runs) for executing the automated test suite; GitHub Issues for defect tracking, with the GitHub Projects board (To Do / In Progress / In Review / Done) used to track defect and test-task status. Pull requests are linked to their issue using “Fixes #ID” or “Closes #ID”, so a merge automatically moves the linked issue to Done.

Test Data: a default (empty/missing) high-score file, a valid pre-populated high-score file, and a deliberately corrupted/malformed high-score file.

7. Test Schedule

Given the team's available time of roughly 1–2 days per week, testing is integrated into each implementation sprint rather than treated as a single late-stage phase:

Milestone	Timing
Unit test design alongside each module's implementation	Same sprint as the module (Sprints 1–5, per SRS-aligned plan)
Integration testing (module-to-module)	End of each sprint, once two or more modules are connected
System / end-to-end testing	Final implementation sprint
Security validation (Section 5.1)	Alongside Persistence Manager implementation and again before final submission
Test summary and defect closure	Before Final Report & Demo deliverable

8. Test Deliverables

- This Test Plan (STP) document
- Test cases (Section 13.1), manual and automated
- Automated unit/integration test source (GoogleTest)
- Test execution results (Actual Result / Pass-Fail, filled in during Implementation & Testing)
- Defect log (tracked via GitHub Issues, with status visible on the GitHub Projects board)
- Final Test Summary Report (part of the Final Report deliverable)

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead / Movement & Session Testing	Rohil Thimmanahalli Girish	Prepare/coordinate the test plan; design and execute test cases for Game Session Management and Snake Movement
Test Engineer / Gameplay Testing	Sai Harshith R	Design and execute test cases for Food/Growth/Scoring and Collision/Game Over; log defects
Test Engineer / Persistence Testing	Sarayu Mayakonda	Design and execute test cases for Pause/Resume, Difficulty and High-Score Persistence, including security validation
Product Owner / Sign-off	Course Instructor	Approve test results and sign off on submission readiness

10. Risks and Mitigation

Risk	Mitigation
Limited weekly team availability (1–2 days/week) leaves insufficient time for full test execution.	Testing is spread across sprints (Section 7) rather than left to the end; High-priority FRs are tested first.
Corrupted/missing high-score file test fixtures are hard to reproduce consistently.	Prepare and commit fixed test-data files (Section 6) to the repository so all three members use identical fixtures.
Core gameplay logic accidentally becomes coupled to SFML rendering, blocking unit tests.	Enforced by the SAD's MVC component boundaries (Game Logic has no direct SFML dependency).
Defects are not consistently reflected in project status.	Use GitHub Issues for every defect and the GitHub Projects board (To Do / In Progress / In Review / Done) to track it; link pull requests to issues with “Fixes #ID”/“Closes #ID” so merges update status automatically.

11. Assumptions & Dependencies

- A locally buildable SFML 2.6 development environment is available to all three team members.
- Test fixtures (valid/corrupted high-score files) will be committed to the repository before persistence testing begins.
- GoogleTest is available via the team's package manager or as a vendored dependency.
- GitHub Issues and the GitHub Projects board are available to all three members for defect and work-item tracking.

12. Suspension & Resumption Criteria

Suspend testing if: the build does not compile/run at all, or a change blocks more than 30% of planned test cases from executing.

Resume testing if: the blocking defect is fixed and the build is confirmed runnable by at least one team member.

13. Test Case Management & Traceability

The SRS Appendix C Requirement Traceability Matrix maps each requirement to its architecture reference, design reference, code file and test case ID. Representative examples:

SRS Requirement	Test Case ID(s)
FR-MOV-004 (block 180° reversal)	TC-02
FR-FGS-002/003 (food consumption/growth)	TC-03
FR-CGS-001 (wall collision)	TC-04
FR-HSC-004/005 (persist / recover high score)	TC-08, TC-09
NFR-PERF-001 (elapsed-time movement)	TC-10

13.1 Detailed Test Cases

The following test cases give functional and non-functional coverage across all seven modules, satisfying the minimum of 7–10 test cases required at this stage. Additional test cases will be added incrementally during Implementation & Testing as each module is built.

TC ID	Requirement	Description	Steps (summary)	Expected Result	Priority
TC-01	FR-MOV-002	Snake moves one cell per game update	Start session; observe snake position over 3 updates	Snake advances exactly one grid cell per update, in the current direction	High
TC-02	FR-MOV-004	Reject 180° reversal	Move Right; immediately press Left	Direction change is rejected; snake continues Right	High
TC-03	FR-FGS-002/003/004	Food consumption grows snake and score	Move snake head onto food cell	Snake length +1, score increases by fixed value, new food spawns	High

TC ID	Requirement	Description	Steps (summary)	Expected Result	Priority
TC-04	FR-CGS-001	Wall collision ends session	Move snake into the grid boundary	Session transitions to game-over; final score displayed	High
TC-05	FR-CGS-002	Self-collision ends session	Grow snake to length ≥ 4 ; steer head into its own body	Session transitions to game-over; final score displayed	High
TC-06	FR-PRS-001/002/003	Pause preserves state, resume continues it	Pause mid-session; wait; resume	Snake position, score and food unchanged across pause/resume	Medium
TC-07	FR-DIF-001/002	Each difficulty applies a distinct speed	Start a session at Easy, then Medium, then Hard	Measured tick interval differs consistently across the three levels	High
TC-08	FR-HSC-003/004	New high score is persisted	Finish a session with score > stored high score	Stored high score file is updated to the new value	High
TC-09	FR-HSC-005, SEC-001	Corrupted high-score file handled gracefully	Replace high-score file with malformed data; launch app	Application starts normally; high score defaults to 0; no crash	High
TC-10	NFR-PERF-001	Movement speed is frame-rate independent	Run the same session at two different rendering frame rates	Snake advances at the same real-time speed regardless of frame rate	Medium

14. Test Metrics & Reporting

Metrics collected: percentage of test cases executed, percentage passed/failed, defect count by severity, requirement coverage (percentage of SRS FR/NFR IDs with at least one linked test case).

Reports: brief status update after each sprint's testing (see Section 7); a Final Test Summary Report compiled as part of the Final Report & Demo deliverable.

15. Approvals

Role	Name	Signature / Date
QA Lead	Rohil Thimmanahalli Girish	
Test Engineer	Sai Harshith R	
Test Engineer	Sarayu Mayakonda	
Product Owner		